Vyreel — Pilot Deployment Notes

From WhatsApp-first to an email-delivered pilot: what we built, what we decided, and what's left.

`Generated June 15, 2026 · scope: 20–30 creator paid pilot, AI/Tech niche`

Goal for the session: get Vyreel ready to launch a pilot for real creators as fast as possible, without over-engineering the infrastructure.

1. What Vyreel is (as deployed)

A daily "content intelligence" service for Instagram creators. Three parts share one SQLite database (`vyreel.db`):

- **Onboarding web app** (`onboarding/app.py`) — Flask signup form: handle, WhatsApp number, email, timezone, competitors.
- **Scheduler daemon** (`scheduler.py`) — runs data collectors every 6h; generates & delivers each creator's brief at 6am in *their* timezone.
- **CLI** (`main.py`) — manual `init` / `collect` / `backfill` / `brief` commands.

Pipeline: collect signals (Reddit, News/RSS, YouTube, Google Trends, Instagram) → score the top topic → Groq LLM writes the brief → WeasyPrint renders a PDF → deliver to the creator.

2. The key decision: email delivery for the pilot

The original delivery channel was the WhatsApp Cloud API. We deliberately switched the pilot to **email** instead.

> **Why:** WhatsApp business-initiated messages (a scheduled 6am brief) legally require Meta Business verification, a permanent access token, and a *pre-approved* message template — days of setup with external approval lead times. Email needs none of that, the Gmail credentials were already wired into the project, and the goal of a pilot is to prove the briefs are *valuable*, not to perfect the delivery channel. A *paid* pilot also offsets email's weaker open rates: paying users actively look for what they bought.

> **Important:** the WhatsApp number is still a **required** onboarding field, and the WhatsApp sender code is left fully intact. That means we can switch WhatsApp back on later **without re-onboarding a single creator.**

3. Major code changes

Change	File(s)	Why
New email sender	`delivery/email.py` (new)	Sends the brief PDF as an attachment via Gmail SMTP. Marks the brief `sent_at` in the DB, mirroring the WhatsApp sender so behaviour is consistent.

Re-pointed delivery	`main.py`, `scheduler.py`	Both the manual `brief` command and the 6am scheduled job now call the email sender instead of WhatsApp.
Email now required	`onboarding/app.py`, `templates/index.html`	Email was optional; it's now the delivery channel, so it's validated and required at signup.
WhatsApp untouched	`delivery/whatsapp.py`	Left as-is, ready to re-enable later. No code was deleted.
Real credentials	`.env`	Replaced placeholder Gmail values with the real sender (`getvyreel@gmail.com`) and a valid 16-char Gmail App Password.

4. Verification — proven live end-to-end

We ran the real pipeline against the test creator `@stics.ai`:

- Collectors pulled fresh data: **News (5)**, **YouTube (31)**, **Trends (10)** signals stored.
- Scorer picked top topic **"Anthropic"** (confidence 44/100).
- Groq generated the brief prose; WeasyPrint rendered the PDF.
- Final result: `[email] sent to rudesword111@gmail.com` — DELIVERED

The first attempt failed with Gmail `535 BadCredentials` — diagnosed as a placeholder/ wrong-type password. **Lesson worth keeping:** Gmail SMTP requires a 16-char *App Password* (2-Step Verification must be on), never the normal account password.

5. Considerations & known risks

Item	Status	Detail
FLASK_SECRET	FIX BEFORE LIVE	Still the default `change_this_to_a_random_string`. It signs session cookies on the public signup form — a guessable value is a security hole. Replace with random text.
Reddit collector	BLOCKED	Every subreddit returns `403 Blocked` (unauthenticated scraping wall). Briefs still generate but with weaker signal. Not a launch blocker.
Instagram collector	OFF	`INSTAGRAM_USER/PASS` not set, so competitor/hashtag data isn't pulled. Same scraping-wall risk as Reddit, but harder at scale. Briefs currently lean on YouTube + Trends + News.
Low confidence scores	EXPECTED	A side effect of Reddit blocked + Instagram off — scores read low (e.g. 44/100) until those sources return. `--force` overrides the send threshold for testing.
Account password in chat	ROTATE	The Gmail *account* password was pasted during setup; recommend changing it. The App Password (revocable, scoped) is what the app actually uses.

6. Hosting note (not yet done)

For 20–30 users, SQLite on one always-on machine is plenty — no need for a complex stack. The only real requirement is that the box stays awake so 6am sends fire (a sleeping laptop = no briefs). A small VM or a managed PaaS both work; the deploy artifacts (start scripts / container config) were deferred pending the hosting choice.

7. What's left to launch

- Set a real `FLASK_SECRET`.
- Run the two processes: `python main.py onboard` (signup form) and `python main.py schedule` (6am delivery daemon).
- Decide where to host so the scheduler stays always-on.
- Open signups. (Optional during pilot: restore Reddit/Instagram signal sources to lift brief quality.)